1. Introduction

The rapid expansion of digital infrastructure has made computer networks a primary target for malicious actors. Global cybercrime damages are projected to exceed ten trillion dollars annually in 2025-2026, with India seeing sharp rises in DDoS and ransomware attacks targeting banking, healthcare, and government networks.

An Intrusion Detection System (IDS) monitors network traffic for unauthorized access or malicious behaviour. Signature-based systems match traffic against known attack patterns; anomaly-based systems flag deviations from a learned baseline. Signature-based approaches fail against zero-day exploits, driving research into ML-based detection capable of generalizing to unseen attacks.

A persistent gap exists between academic IDS research and practical deployment. Most published systems evaluate on static datasets without addressing real-time operational pipelines, continuous alerting, or analyst-facing interfaces limiting operational value. A further challenge is severe class imbalance: benign flows dominate real traffic, and models that ignore this risk high false-negative rates on minority attack classes.

This project is motivated by CERT-In mandates for faster incident detection and the expanded network perimeter from hybrid cloud and remote work environments.

Objectives:

1. Train a Random Forest + XGBoost ensemble on CIC-IDS2017 for multi-class intrusion detection.
2. Implement a real-time alert mechanism upon detection of anomalous flows.
3. Build a pipeline for live/replayed traffic: feature extraction → classification → dashboard.
4. Comparatively evaluate RF, XGBoost, and the ensemble on accuracy, precision, recall, and F1.

NIDAS proposes a soft-voting ensemble with two operational modes: a demo mode replaying CIC-IDS2017 records and a live capture mode using a custom CICFlowMeter implementation. Each flow is classified into: Normal Traffic, DoS, DDoS, Port Scanning, or Brute Force. Detected anomalies trigger timestamped alerts with predicted attack type and confidence score. A React JS dashboard surfaces live flow classifications; a FastAPI backend handles communication between the detection engine and UI.

The key contribution is end-to-end integration of ensemble detection with real-time alerting and dashboard visualization operationally actionable, not just accurate.

2. Supporting Literature

2.1 Literature Review

Paper 1 - Ness S, Eswarakrishnan V, Sridharan H, Shinde V, Janapareddy N V P, & Dhanawat V. (2025). Anomaly detection in network traffic using advanced machine learning techniques. IEEE Access, vol. 13, pp. 16133-16149.

This paper investigates various machine learning models for detecting anomalies in network traffic, focusing on overcoming challenges like class imbalance and feature complexity. Accurate anomaly detection serves as a proactive measure to safeguard network integrity and identify dynamic security risks.

The authors utilize five machine learning algorithms: Isolation Forest, Naive Bayes, XGBoost, LightGBM, and Support Vector Machine (SVM). The study relies on the KDDCup'99 dataset, which includes normal activity and multiple attack types like DoS, Probe, Remote to Local (R2L), and User to Root (U2R). Data preprocessing involves the Synthetic Minority Over-sampling Technique (SMOTE) for balancing classes, label encoding for categorical features, and Principal Component Analysis (PCA) for feature extraction and dimensionality reduction.

The models were evaluated using 5-fold cross-validation with metrics including accuracy, precision, recall, and F1-score. LightGBM achieved the highest test accuracy (1.0) and F1-score (0.85). SVM also demonstrated strong performance with a test accuracy of 0.85 and F1-score of 0.86. Conversely, Isolation Forest performed poorly with a test accuracy of 0.40, struggling with the dataset's complexity.

The study concludes that advanced ensemble and gradient boosting methods like LightGBM and XGBoost, alongside SVM, offer robust detection capabilities for network traffic anomalies. Future work suggests integrating deep learning techniques like CNNs and RNNs, using ensemble methods to reduce false positives, and applying models to newer datasets like CICIDS and NSL-KDD.

Reference: https://doi.org/10.1109/ACCESS.2025.3526988

TITLE	Anomaly Detection in Network Traffic Using Advanced Machine Learning Techniques.
AREA OF WORK	Evaluating and optimizing supervised and unsupervised machine learning models to detect anomalies in network traffic, specifically addressing data imbalance and feature complexity.
DATASET	KDDCup'99 dataset containing normal traffic and multiple attack types (DoS, Probe, R2L, U2R). The dataset was balanced using SMOTE.
METHODOLOGY/ STRATERGY	Data preprocessing involved handling missing values, label encoding, standardizing via z-score normalization, and feature extraction using PCA. Five ML models were trained with hyperparameter tuning and evaluated using 5-fold cross-validation with an 80:20 train-test split.
ALGORITHM	• Isolation Forest • Naive Bayes • XGBoost • LightGBM • Support Vector Machine (SVM)
RESULT/PRECISION	• Best performer: LightGBM (Test Accuracy = 1.0, F1-score = 0.85). • SVM achieved Test Accuracy = 0.85 and F1-score = 0.86. • XGBoost achieved Test Accuracy = 0.83 and F1-score = 0.84. • Isolation Forest performed poorly with Test Accuracy = 0.40.
ADVANTAGES	Provides a comprehensive comparison of classical ML models on network anomaly detection. Highlights the efficacy of gradient boosting methods and effectively utilizes SMOTE and PCA to handle class imbalance and high dimensionality.

FUTURE PROPOSAL	Recommends integrating deep learning techniques (CNNs, RNNs), using ensemble methods to reduce false positives, exploring unsupervised approaches for zero-day attacks, and validating models on newer datasets like CICIDS and NSL-KDD.

Table 2.1 Summary of Paper 1

Paper 2 - Fathima AHN, Ibrahim SPS, Khraisat A. Enhancing Network Traffic Anomaly Detection: Leveraging Temporal Correlation Index in a Hybrid Framework. IEEE Access. 2024 Sep 11;12:136805-136824.

This paper introduces the Temporal Metric-Driven GRU Embedded Generative Neural Network (TMG-GRU-VAE) to improve the identification of network traffic irregularities. Traditional models often emphasize detecting network attacks but struggle to capture the dynamic and temporal aspects of network traffic, making it difficult to detect subtle behavioral shifts.

The proposed methodology combines Gated Recurrent Units (GRU) with Variational Autoencoders (VAE) to effectively learn normal temporal patterns in sequence data. Additionally, the authors propose a novel Temporal Correlation Index (TCI) score to assess temporal behavior and establish dynamic anomaly detection thresholds. The study utilizes the CIC-IDS-2017 and CIC-IDS-2018 datasets, splitting the preprocessed data into 70% training and 30% testing subsets.

The models were evaluated using accuracy, precision, recall, and F1-score, alongside false alarm rates. The TMG-GRU-VAE model combined with TCI achieved significant reductions in False Positives (FP). Improvements in FP reduction ranged from 7.2% to 12.9% for the CIC-IDS-2017 dataset, and 7.1% to 14.1% for the CIC-IDS-2018 dataset. The GRU-VAE with TCI achieved the highest True Positive rates and the lowest FP rates.

The research demonstrates that integrating temporal correlation metrics with hybrid deep learning frameworks substantially reduces false alarms and improves overall anomaly detection accuracy. Future work aims to develop explainable AI models to enhance interpretability and confidence in detection results, particularly for addressing zero-day threats.

Reference: https://doi.org/10.1109/ACCESS.2024.3458903

TITLE	Enhancing Network Traffic Anomaly Detection: Leveraging Temporal Correlation Index in a Hybrid Framework

AREA OF WORK	Developing a hybrid deep learning framework (TMG-GRU-VAE) combined with a Temporal Correlation Index (TCI) to capture temporal dependencies and improve network anomaly detection.
DATASET	CIC-IDS-2017 and CIC-IDS-2018 datasets containing comprehensive network intrusion scenarios and temporal features.
METHODOLOGY/ STRATERGY	The approach integrates GRUs into a VAE to learn sequence-based traffic behaviors. A novel TCI metric is computed by comparing original and reconstructed sequences to define dynamic anomaly thresholds. The data was preprocessed and split into a 70:30 train-test ratio.
ALGORITHM	• Temporal Metric-Driven GRU Embedded Generative Neural Network (TMG-GRU-VAE) • LSTM-VAE • Temporal Correlation Index (TCI) thresholding
RESULT/PRECISION	• GRU-VAE with TCI achieved the best performance with the lowest False Positive rate (7.2% on CIC-IDS-2017, 7.1% on CIC-IDS-2018). • Precision improved significantly, from 66.3% to 69.4% in specific configurations. • The TCI approach consistently outperformed standalone LSTM-VAE and GRU-VAE models.
ADVANTAGES	Effectively manages both feature complexities and temporal shifts in network traffic. Drastically reduces false alarm rates while identifying subtle behavioral changes over time.
FUTURE PROPOSAL	Focuses on developing explainable AI models to understand why specific traffic is flagged as anomalous, and extending testing to larger-scale environments to assess scalability and zero-day attack resilience.

Table 2.2 Summary of Paper 2

Paper 3 - Ji C, Liu H, Dai W. Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Two-Stage Feature Fusion. IEEE Access. 2025 Feb 4;13:27310-27324.

The paper proposes GRU-ResCBANet, a deep learning hybrid model tailored for network traffic anomaly detection. The model addresses issues such as low detection rates for minority attack classes and high false alarm rates by employing a parallel two-stage feature extraction and fusion technique.

In the first stage, the ResCBANet module uses an improved residual network enhanced with a Convolutional Block Attention Module (CBAM) to extract spatial features and channel attention. In parallel, a Gated Recurrent Unit (GRU) module captures temporal dependencies. These features are fused and passed to a fully connected classification layer using a Softmax activation function. The model was evaluated on the NSL-KDD, UNSW-NB15, and CICIDS2017 datasets, which were preprocessed and reduced in dimensionality using ANOVA for feature selection.

GRU-ResCBANet delivered state-of-the-art results. For binary classification, it achieved accuracies of 99.70% on NSL-KDD, 99.16% on UNSW-NB15, and 99.83% on CICIDS2017. For multi-class classification, accuracies reached 99.69%, 98.23%, and 99.79%, respectively. The model consistently outperformed eight other comparison models, including SVM, RF, 2D-CNN, and CNN-LSTM.

The study concludes that parallel integration of GRU and ResCBANet successfully captures comprehensive feature representations, significantly boosting anomaly detection accuracy. Future directions include exploring model compression for edge computing, applying oversampling strategies for class imbalance (especially in UNSW-NB15), and integrating federated learning for privacy-preserving distributed security.

Reference: https://doi.org/10.1109/ACCESS.2025.3538319

TITLE	Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Two-Stage Feature Fusion.
AREA OF WORK	Improving network traffic anomaly detection accuracy and reducing false alarm rates using a hybrid deep learning model that fuses spatiotemporal and channel features.
DATASET	NSL-KDD, UNSW-NB15, and CICIDS2017 datasets. Data was transformed into 3D structured inputs after dimensionality reduction via ANOVA.

METHODOLOGY/ STRATERGY	The model uses a parallel two-stage architecture. Stage one uses ResCBANet (CNN + Residual Units + CBAM) for spatial/channel features. Stage two uses GRUs for temporal features. The outputs are fused and classified using a fully connected layer with a Softmax activation.
ALGORITHM	• GRU-ResCBANet • Residual Convolutional Block Attention Network (ResCBANet) • Gated Recurrent Units (GRU)
RESULT/PRECISION	• Binary classification accuracies: 99.70% (NSL-KDD), 99.16% (UNSW-NB15), and 99.83% (CICIDS2017). • Multi-class classification accuracies: 99.69% (NSL-KDD), 98.23% (UNSW-NB15), and 99.79% (CICIDS2017). • Achieved the highest F1-scores compared to 8 baseline models.
ADVANTAGES	The parallel two-stage strategy efficiently leverages multidimensional characteristics of traffic data. It excels in detecting both frequent and complex attack types while maintaining high stability and accuracy.
FUTURE PROPOSAL	Suggests investigating model pruning/quantization for resource-constrained edge devices, addressing the class imbalance in UNSW-NB15 using techniques like SMOTE, and applying federated learning for privacy.

Table 2.3 Summary of Paper 3

2.2 Literature Review Summary

TITLE	SUMMARY
Ness S, Eswarakrishnan V, Sridharan H, Shinde V, Janapareddy NVP, Dhanawat V. Anomaly Detection in Network Traffic Using Advanced	This study evaluates multiple machine learning models : Isolation Forest, Naïve Bayes, XGBoost, LightGBM, and SVM for network traffic anomaly detection using the KDDCup'99 dataset. By utilizing SMOTE to handle class imbalance and

Machine Learning Techniques. IEEE Access. 2025 Jan 8;13:16133-16149.	PCA for dimensionality reduction, the authors demonstrated that LightGBM and SVM achieved the highest accuracy and F1-scores, whereas Isolation Forest struggled with the dataset's complexity. The findings confirm the effectiveness of advanced ensemble and gradient boosting methods for robust network security.
Fathima AHN, Ibrahim SPS, Khraisat A. Enhancing Network Traffic Anomaly Detection: Leveraging Temporal Correlation Index in a Hybrid Framework. IEEE Access. 2024 Sep 11;12:136805-136824.	This paper introduces a hybrid deep learning framework (TMG-GRU-VAE) that combines Gated Recurrent Units with Variational Autoencoders to capture temporal dependencies in network traffic. It also proposes a novel Temporal Correlation Index (TCI) to establish dynamic detection thresholds. Evaluated on the CIC-IDS-2017 and CIC-IDS-2018 datasets, the method drastically reduced false positive rates, proving that incorporating temporal correlation metrics significantly improves the detection of subtle anomaly shifts over time.
Ji C, Liu H, Dai W. Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Two-Stage Feature Fusion. IEEE Access. 2025 Feb 4;13:27310-27324.	The authors propose GRU-ResCBANet, a hybrid model that uses parallel two-stage feature extraction to improve detection accuracy and lower false alarms. The first stage uses a CNN with Residual Units and a Convolutional Block Attention Module (CBAM) for spatial and channel features, while the second utilizes GRUs for temporal features. Tested across NSL-KDD, UNSW-NB15, and CICIDS2017 datasets, the model achieved state-of-the-art accuracy in both binary and multi-class classification, successfully capturing multidimensional traffic characteristics.

Table 2.4 Literature Summary

2.3 Findings and Proposals

Findings

The reviewed literature establishes several consistent findings. Ensemble and fusion-based approaches uniformly outperform single-model classifiers in network traffic anomaly detection, across both traditional ML and deep learning paradigms. The CIC-IDS2017 dataset is the dominant benchmark in current IDS research and is the standard against which results from this project are comparable. Temporal features, particularly IAT-based features, carry significant discriminative information that improves detection of attack types with characteristic timing signatures. Despite the progress in detection accuracy, none of the three reviewed papers implement a real-time operational pipeline; all evaluate models in an offline, batch mode.

Identified Gap

The common gap across all three reviewed papers is the absence of a real-time alert mechanism. Each paper treats intrusion detection as a classification task on a static dataset, with no consideration of how detected anomalies are surfaced to an operator as they occur. This gap is practically significant: a model with high batch accuracy that cannot generate timely alerts provides no operational advantage over a post-hoc log analysis.

Proposal

This project proposes to close this gap by integrating a real-time alerting mechanism directly into the ensemble detection pipeline. The proposed system extends the soft-voting RF + XGBoost ensemble validated by Ness et al. with a FastAPI backend that classifies incoming flows in real time and triggers alert notifications upon detection of anomalous traffic. A React JS dashboard surfaces these alerts to the operator with predicted attack category and ensemble confidence score. This integration of detection and alerting constitutes the novel contribution of this project over the reviewed literature.

3. System Analysis

3.1 Analysis of the Dataset

3.1.1 About the Dataset

The CIC-IDS2017 dataset, developed by the Canadian Institute for Cybersecurity, is one of the most widely cited benchmark datasets in network intrusion detection research. It was generated using a realistic network environment over five days (Monday through Friday) in July 2017, capturing both benign background traffic and a diverse range of attack scenarios. Traffic was captured using CICFlowMeter, which extracts over 80 bidirectional flow-level features from raw packet captures.

The dataset used in this project consists of the eight raw CICFlowMeter CSV files sourced from the Canadian Institute for Cybersecurity (CIC) at cicresearch.ca, combined into a single dataframe of 2,830,743 flow records and 80 columns, 78 numerical feature columns, one categorical label column (Label), and one metadata column (source_file) added during merging. The dataset originally contains 15 distinct raw labels. Of these, 9 labels are consolidated into five traffic categories: BENIGN is mapped to Normal Traffic; DoS Hulk, DoS GoldenEye, DoS slowloris, and DoS Slowhttptest are grouped as DoS; DDoS is retained as DDoS; FTP-Patator and SSH-Patator are grouped as Brute Force; and PortScan is mapped to Port Scan. The remaining six labels Bot, Web Attack – Brute Force, Web Attack – XSS, Web Attack – Sql Injection, Infiltration, and Heartbleed are excluded from the dataset as they fall outside the five target classification categories, either due to insufficient representation or scope limitations. All features are numeric and flow-level, making the dataset directly compatible with tabular machine learning methods such as Random Forest and XGBoost.

3.1.2 Explore the Dataset

The dataset comprises 2,830,743 records across 78 numeric feature columns. Feature columns are of numeric type (float64 or int64), with two non-numeric columns, Label and source_file, excluded from feature analysis. Two columns, Flow Bytes/s and Flow Packets/s, contain 2,857 infinite values each, arising from flows with near-zero duration. These are the only data quality issues detected and are addressed in the cleaning step.

The 78 features span several categories of network flow statistics: packet length statistics (minimum, maximum, mean, standard deviation for both forward and backward directions), inter-arrival time (IAT) statistics for the overall flow and for forward and backward sub-flows, flag counts (FIN, SYN, RST, PSH, ACK, URG, CWE, ECE), flow-level throughput metrics (Flow Bytes/s, Flow Packets/s), TCP window and header length features, subflow statistics,

and active/idle timing features. Eight features, Bwd PSH Flags, Bwd URG Flags, and all six bulk rate columns, are entirely constant across the dataset and carry no discriminative information, making them candidates for removal during preprocessing.

Key observations from the descriptive statistics are as follows. Flow Duration ranges from near zero to approximately 120 million microseconds with a median of 31,298 μs, indicating a heavily right-skewed distribution where most flows are short-lived. Flow Bytes/s has a mean of approximately 1.49 million bytes per second with a standard deviation of 25.96 million, and notably contains negative minimum values, which warrant investigation as they are conceptually unexpected for a rate feature. Packet Length Mean has a dataset-wide mean of 172.1 bytes with a standard deviation of 305.6 bytes. Flag features such as FIN Flag Count, PSH Flag Count, and ACK Flag Count are binary (0 or 1), with mean values of 0.035, 0.297, and 0.316 respectively, indicating their sparse but discriminative nature. Active and Idle features have medians of zero, meaning the majority of flows do not exhibit activity/idle cycles, but non-zero values in a subset of flows carry significant discriminative information for certain attack patterns. Additionally, 307,730 duplicate rows are present in the combined dataset, representing approximately 10.9% of all records, and must be removed prior to train/test splitting to prevent data leakage.

3.2 Data Preprocessing

The raw CIC-IDS2017 dataset, combined from eight source files, contains 2,830,743 flow records across 78 numeric features. The cleaning pipeline addresses the following data quality issues in sequence.

Eight feature columns are entirely constant across the dataset: Bwd PSH Flags, Bwd URG Flags, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, and Bwd Avg Bulk Rate. These are removed as they carry no discriminative information, leaving 70 features.Two rate-based features, Flow Bytes/s and Flow Packets/s, contain 2,857 infinite values each, arising from flows with near-zero duration. Infinite values are converted to NaN and the affected rows are dropped, reducing the dataset to 2,823,693 records.

The dataset contains 307,730 duplicate flow records, representing approximately 10.9% of all records. These are removed prior to any train/test splitting to prevent data leakage, reducing the dataset to 2,515,963 records. The final class distribution after cleaning is as follows: Normal Traffic (2,094,458 records, 83.3%), DoS (193,647, 7.7%), DDoS (128,014, 5.1%), Port Scan (90,694, 3.6%), and Brute Force (9,150, 0.4%). The severe imbalance between Normal Traffic

and minority attack classes, particularly Brute Force at 0.4% of the dataset, directly motivates the manual sample-weight correction applied during model training. The label column is encoded using a LabelEncoder, mapping the five class names to integer indices required by the scikit-learn and XGBoost APIs.

3.2.2 Analysis of Feature Variables

The dataset contains 78 numeric feature columns after combining the eight source files. Following the cleaning and feature selection pipeline, 46 features are retained for model training. These can be grouped into the following categories based on the aspect of network flow behaviour they capture.Packet length features capture the size characteristics of packets in both forward (client-to-server) and backward (server-to-client) directions, including maximum, minimum, mean, and standard deviation values. Random Forest feature importance analysis identifies this group as the strongest family of discriminative features, with Bwd Packet Length Mean, Packet Length Std, Bwd Packet Length Std, and Packet Length Variance occupying the top four importance ranks. These features are particularly discriminative for DoS and DDoS attacks, which generate packets of characteristic sizes at high rates.

Inter-Arrival Time (IAT) features measure the time intervals between successive packets within a flow, computed separately for the overall flow, the forward sub-flow, and the backward sub-flow. Mean, standard deviation, maximum, and minimum IAT values are included. As demonstrated by Fathima et al. (2024), temporal features of this nature carry significant discriminative information for attacks that exhibit periodic or bursty timing patterns.

Traffic volume features include Total Length of Fwd Packets, Total Length of Bwd Packets, Total Fwd Packets, and Total Backward Packets, characterizing the overall data volume transferred in each direction. These complement packet length features for detecting volume-based attacks.

Flow-level throughput features include Flow Bytes/s, Flow Packets/s, and Bwd Packets/s, which characterize the volumetric rate of the flow.

TCP flag features record the presence of FIN, PSH, and ACK flags within the flow. These help distinguish connection setup and teardown patterns associated with specific attack types such as port scans.

TCP window and header features include the initial TCP window sizes in both directions, forward and backward header lengths, and minimum segment size, which reflect protocol-level behaviour that varies across attack types.

Active and Idle time features capture the periods of activity and inactivity within a flow. Active Mean, Active Max, Active Min, and Idle Mean are retained after feature selection.

Feature selection was performed in three stages on a representative 50,000-row sample: Kruskal-Wallis statistical testing eliminated two features (CWE Flag Count and RST Flag Count) that showed no statistically significant difference across classes, and Random Forest importance-based selection eliminated a further five low-importance features (Bwd IAT Std, URG Flag Count, Active Std, SYN Flag Count, and Idle Std), arriving at the final 46-feature set. Feature scaling using StandardScaler is applied prior to training to normalize the wide dynamic ranges observed across features.

3.2.3 Analysis of Class Variables

To understand how effectively different network features distinguish between benign traffic and various attack families, a class-separation analysis was conducted on the label-grouped dataset of 2,515,963 records across five traffic classes.

Feature Distribution Across Attack Families (Log1p Scale)

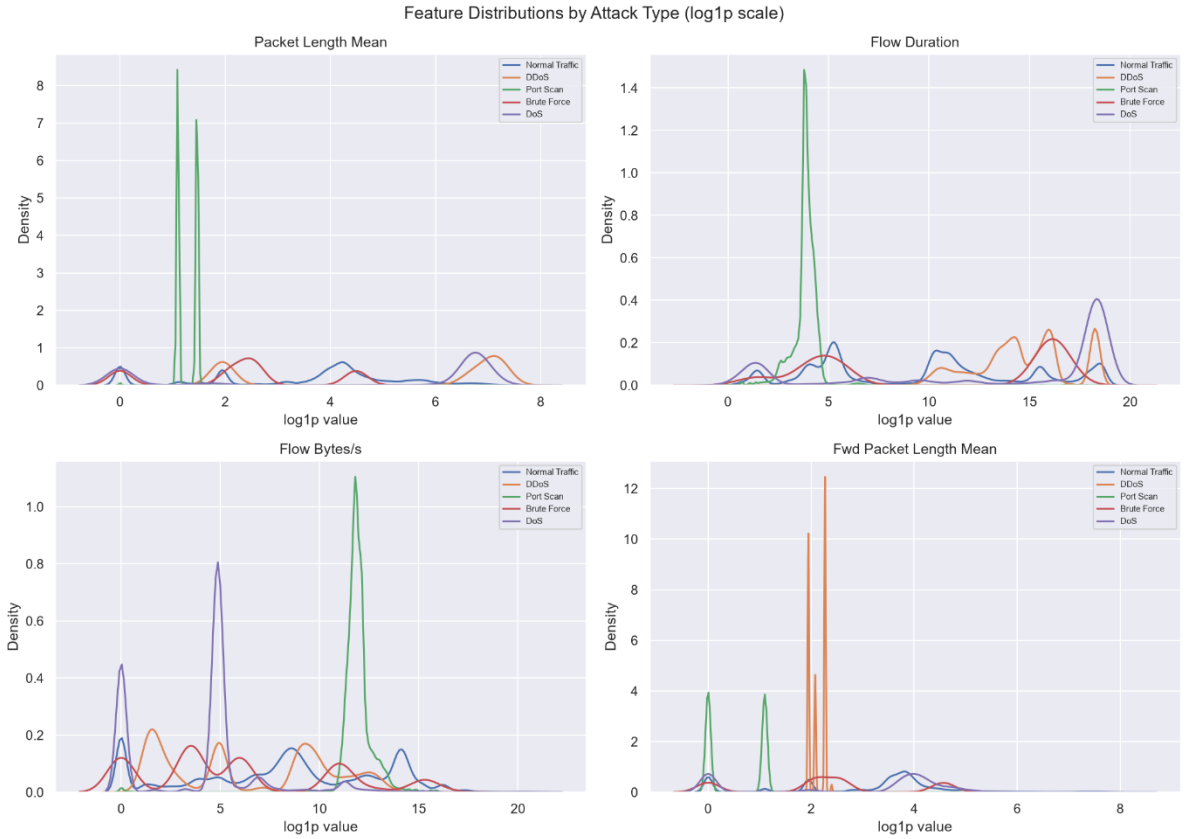


Figure 3.1

Visualizing the features on a logarithmic scale, as shown in Figure 3.1, reveals distinct behavioral patterns across different attack categories:

- **Packet Length Mean:** This shows strong class separation. Port Scan traffic produces an extremely sharp, narrow spike at very low values (log1p ≈ 1.5), consistent with its near-zero mean packet length of 3.25 bytes. DDoS traffic concentrates at a high value

with a tight distribution, reflecting its mean of 736.9 bytes. DoS is similarly elevated at 565.7 bytes but more spread out. Normal Traffic and Brute Force overlap considerably in the mid-range, with Normal Traffic showing a broad, multimodal distribution.

- **Flow Duration:** This feature shows the clearest separation for Port Scan, which produces a sharp isolated spike at very low durations (mean 82,820 μs), consistent with rapid probe-style connections. DoS flows have the longest durations (mean 5.57×10^7 μs), forming a distinct cluster at the high end. DDoS and Normal Traffic overlap substantially across the mid-to-high range, while Brute Force shows a moderate spread.
- **Flow Bytes/s:** This reveals a sharp, isolated spike for Port Scan at high values ($\log1p \approx 11$), while DoS shows a prominent peak at mid-range values ($\log1p \approx 5$). Normal Traffic is broadly distributed with mass concentrated near zero. DDoS and Brute Force occupy the lower range with considerable overlap with Normal Traffic.
- **Fwd Packet Length Mean:** This feature shows the most visually distinct separation. Port Scan and DDoS both produce extremely narrow, high-density spikes at near-zero and low values respectively, while Brute Force shows a sharp spike around $\log1p \approx 2$. Normal Traffic is broadly distributed. The class means confirm this: Port Scan has a mean of 1.01 bytes, DDoS 7.40 bytes, and Normal Traffic 66.4 bytes.

Class Overlap and Non-Linearity

While certain attack classes exhibit highly specific, isolated signatures in individual features, substantial overlap exists between Normal Traffic and attack classes across the lower ranges of most features. No single feature provides a clean decision boundary across all five classes. The multimodal distributions of Normal Traffic in particular confirm that benign flows span a wide behavioral range, making simple threshold-based detection insufficient. This analysis supports the use of a multi-feature, non-linear ensemble approach capable of exploiting the combined discriminative signal across all 46 selected features.

3.3 Data Visualization

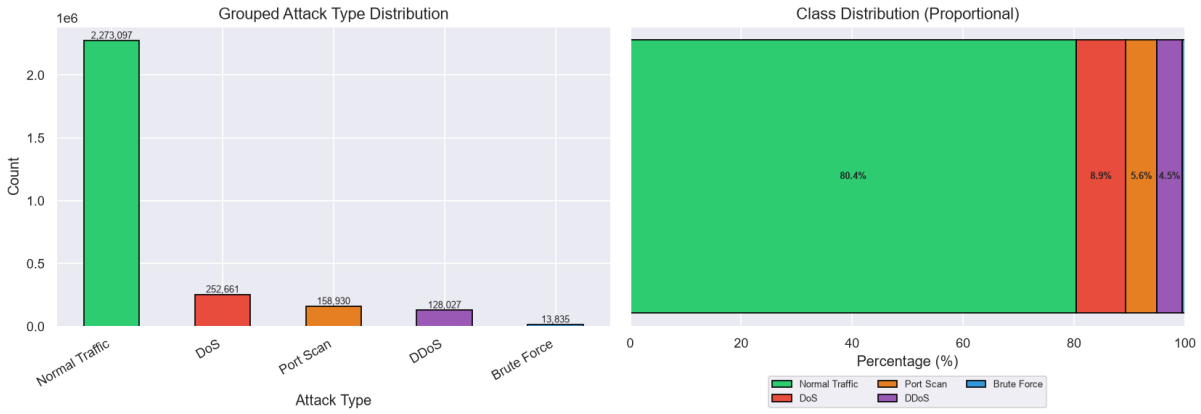


Figure 3.2: Class Distribution

Figure 3.2 presents the class distribution of the CIC-IDS2017 dataset as both a bar chart and a stacked bar chart. The bar chart clearly illustrates the dominance of the Normal Traffic class (2,273,097 samples) relative to all attack categories, making the scale of imbalance immediately apparent. The stacked bar chart further quantifies this disparity: Normal Traffic constitutes 80.4% of all records, while DoS (8.9%), DDoS (4.5%), and Port Scanning (5.6%) together account for most of the remaining traffic. Minority class, Brute Force (0.5%) is barely visible in the stacked bar chart, underscoring the challenge they pose during classification.

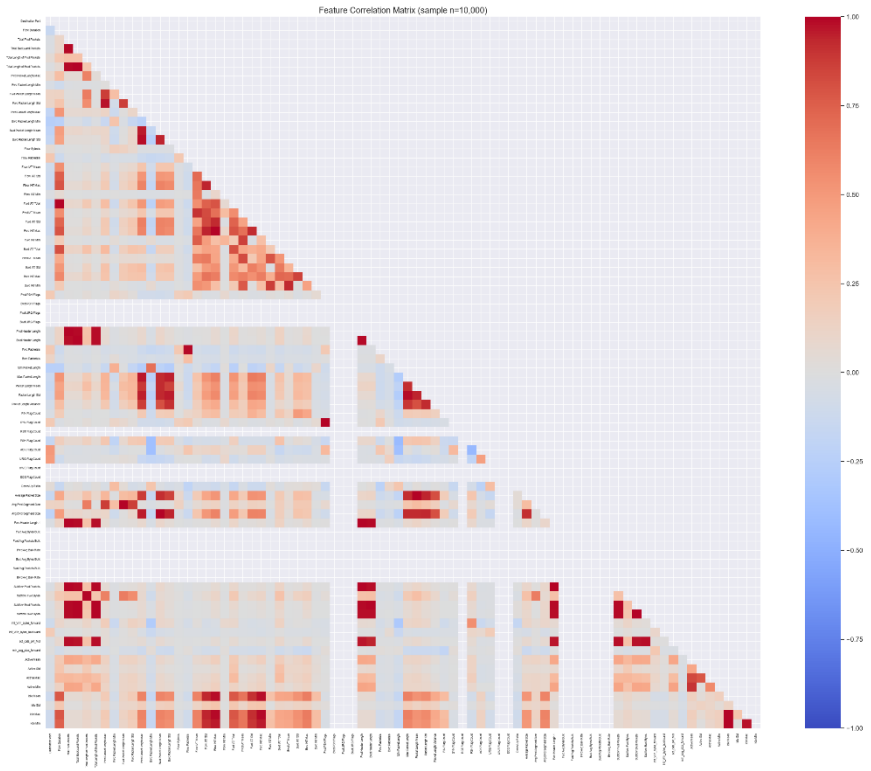


Figure 3.3: Feature Correlation Matrix (sample n=10,000)

Figure 3.3 displays the Pearson correlation matrix computed on a random sample of 10,000 records across the full feature set. The heatmap reveals several distinct clusters of high redundancy, with 27 feature pairs exceeding a correlation threshold of 0.99.

The most striking redundancies are perfect correlations (r = 1.0000): Total Fwd Packets with Subflow Fwd Packets, Total Backward Packets with Subflow Bwd Packets, Total Length of Fwd Packets with Subflow Fwd Bytes, Total Length of Bwd Packets with Subflow Bwd Bytes, Fwd Packet Length Mean with Avg Fwd Segment Size, Bwd Packet Length Mean with Avg Bwd Segment Size, Fwd PSH Flags with SYN Flag Count, and Fwd Header Length with Fwd Header Length.1. These pairs are entirely redundant and one feature from each pair carries no additional information.

Beyond perfect correlations, several near-perfect relationships are observed. Flow Duration and Fwd IAT Total share a correlation of 0.9981, reflecting that total forward inter-arrival time approximates the overall flow duration. Flow IAT Max and Fwd IAT Max are correlated at 0.9975, and Packet Length Mean with Average Packet Size at 0.9976. The Idle feature cluster shows strong internal correlation, with Idle Mean correlating with both Idle Max (0.9924) and Idle Min (0.9927). Header length and subflow packet count features form another redundant cluster, with Total Fwd Packets, Fwd Header Length, Fwd Header Length.1, and Subflow Fwd Packets all mutually correlated above 0.99.

These findings directly informed the correlation-based feature reduction step in the preprocessing pipeline, where one feature from each redundant pair was removed, reducing the feature set from 70 to 53 features prior to statistical and importance-based selection.

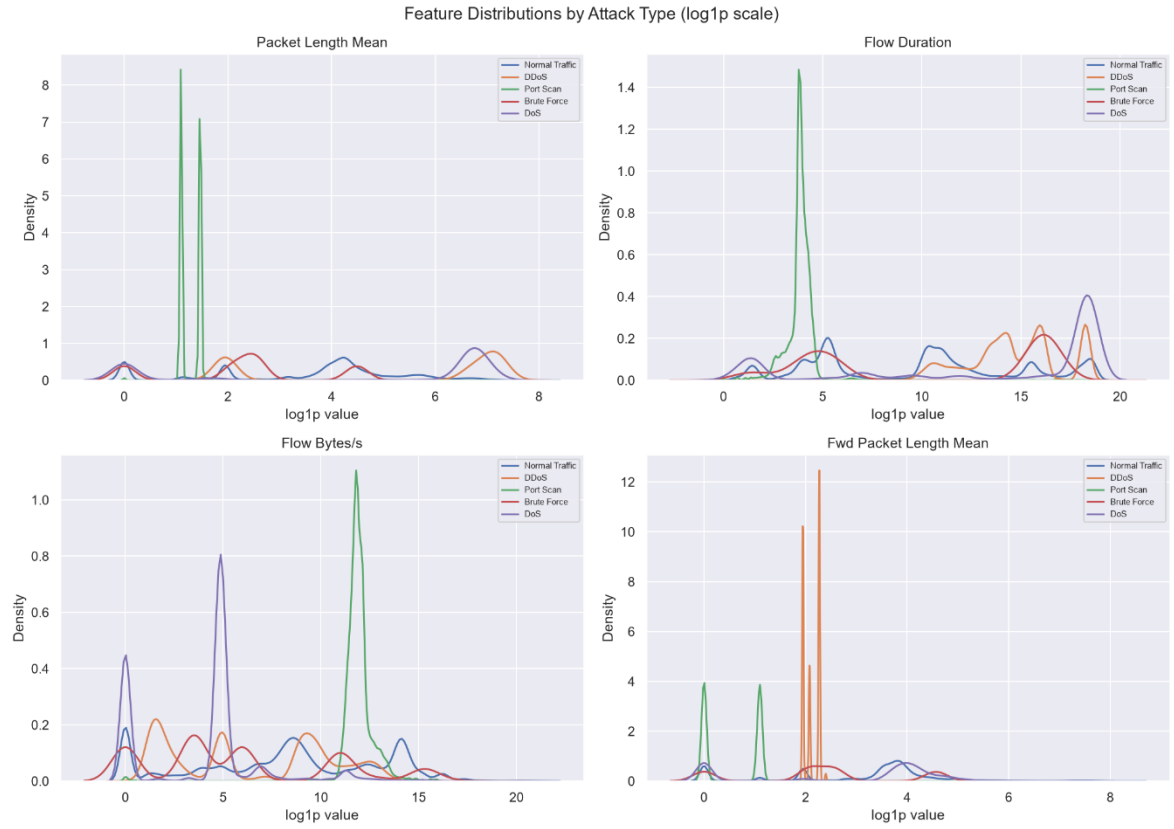


Figure 3.4: Key Feature Distributions by Attack Type

Figure 3.4 shows the density distributions of four key features, Packet Length Mean, Flow Duration, Flow Bytes/s, and Fwd Packet Length Mean, stratified by the five attack classes on a log1p scale. Across all four features, the distributions are heavily right-skewed, with most values concentrated near zero and long tails extending toward high values. Port Scan produces a sharp, isolated spike at very low Packet Length Mean values, while DDoS and DoS cluster at higher values with tighter distributions. Flow Duration shows Port Scan concentrated at very short durations, consistent with rapid probe-style connections, while DoS spans the highest duration range, reflecting sustained flooding behaviour. Flow Bytes/s reveals a distinctive high-value spike for Port Scan, while DoS shows a prominent mid-range concentration and Normal Traffic spreads broadly across lower values. Fwd Packet Length Mean shows the sharpest class separation, with Brute Force and DDoS producing extremely narrow, high-density spikes at low values, clearly distinguishable from the broader Normal Traffic distribution. The visible separation between classes across these features confirms their relevance as discriminative inputs to the ensemble classifier.

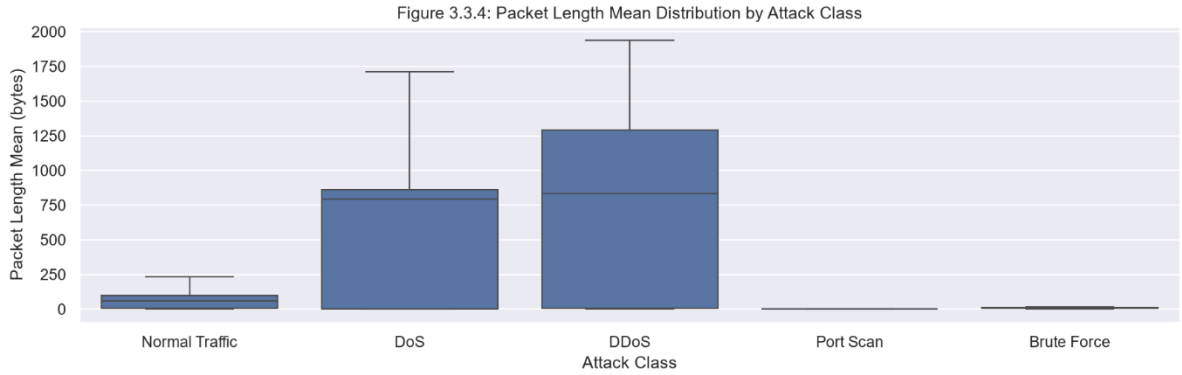


Figure 3.5: Packet Length Mean Distribution by Attack Class

Figure 3.5 presents box plots of Packet Length Mean across the five traffic classes. DDoS and DoS show substantially higher median packet lengths, approximately 800 bytes each, with DDoS exhibiting a wider interquartile range indicating greater variability in payload size. Normal Traffic has a low median of approximately 80 bytes with a moderate spread. Port Scan and Brute Force both have medians near zero, consistent with the short probe packets characteristic of scanning activity and the small credential packets used in brute force attempts. These inter-class differences in packet length make it a strongly discriminating feature for the ensemble model, as confirmed by its top-10 ranking in Random Forest feature importance.

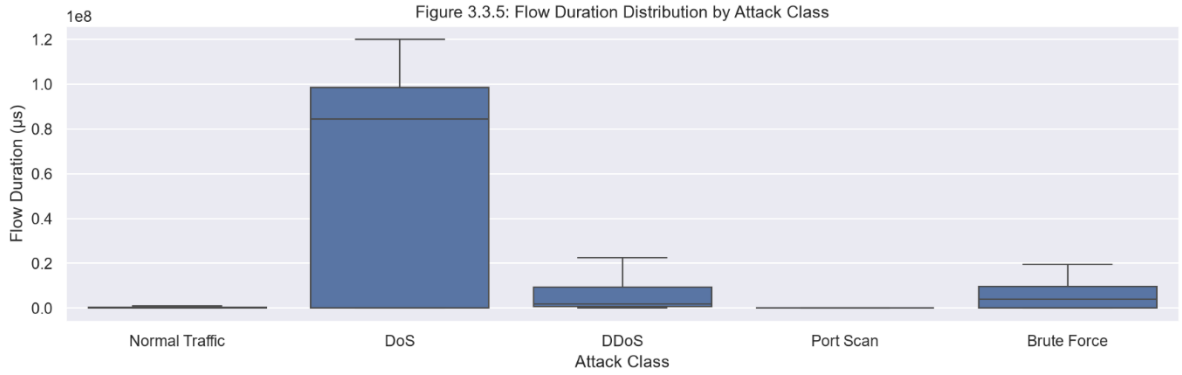


Figure 3.6: Flow Duration Distribution by Attack Class

Figure 3.6 shows box plots of Flow Duration in microseconds across the five traffic classes. DoS flows exhibit by far the longest durations, with the median around 8×10^7 µs and a wide interquartile range extending to approximately 1×10^8 µs, consistent with sustained flooding behaviour that keeps connections open for extended periods. DDoS, Brute Force, and Normal Traffic all have low medians compressed near zero relative to DoS, though with varying spreads. Port Scan shows the shortest and most tightly concentrated durations, corresponding

to its rapid, probe-style connection pattern. Flow Duration thus provides strong discriminative signal, particularly for separating DoS from all other classes.

3.4 Analysis of Algorithm

3.4.1 Detailed Study Report of the Proposed Algorithms

Random Forest

Random Forest is an ensemble learning method that constructs a large number of decision trees during training and outputs the class with the highest aggregated vote (for classification) across all trees. Each tree is trained on a bootstrap sample of the training data (sampling with replacement), and at each split, only a random subset of features is considered as candidates for the split criterion. This combination of data bagging and feature randomization reduces the variance of the ensemble compared to any individual tree and makes the model robust to overfitting, even on high-dimensional datasets such as CIC-IDS2017 with 46 features.

Strengths**:**

Handles mixed feature scales without requiring normalization. Naturally produces feature importance scores that identify the most discriminative flow attributes. Robust to noisy or redundant features, which is important given the high inter-feature correlations observed in the CIC-IDS2017 dataset. Scales efficiently to large datasets via parallel tree construction (n_jobs=-1).

Justification:

- Ness et al. (2025), in a multi-model comparison across Isolation Forest, Naive Bayes, XGBoost, LightGBM, and SVM on network traffic anomaly detection, demonstrated that tree-based ensemble methods consistently outperform single-model classifiers on structured network flow data. This provides peer-reviewed confirmation that Random Forest, as a bagging ensemble, is a well-validated choice for the base learner role in this project.
- Fathima et al. (2024) address high false positive rates in IDS the same practical reliability challenge motivating this project's ensemble approach and validate their hybrid detection framework on the CIC-IDS2017 dataset, the same benchmark used here. The consistent effectiveness of ensemble methods on this dataset directly supports Random Forest's inclusion as a base learner.

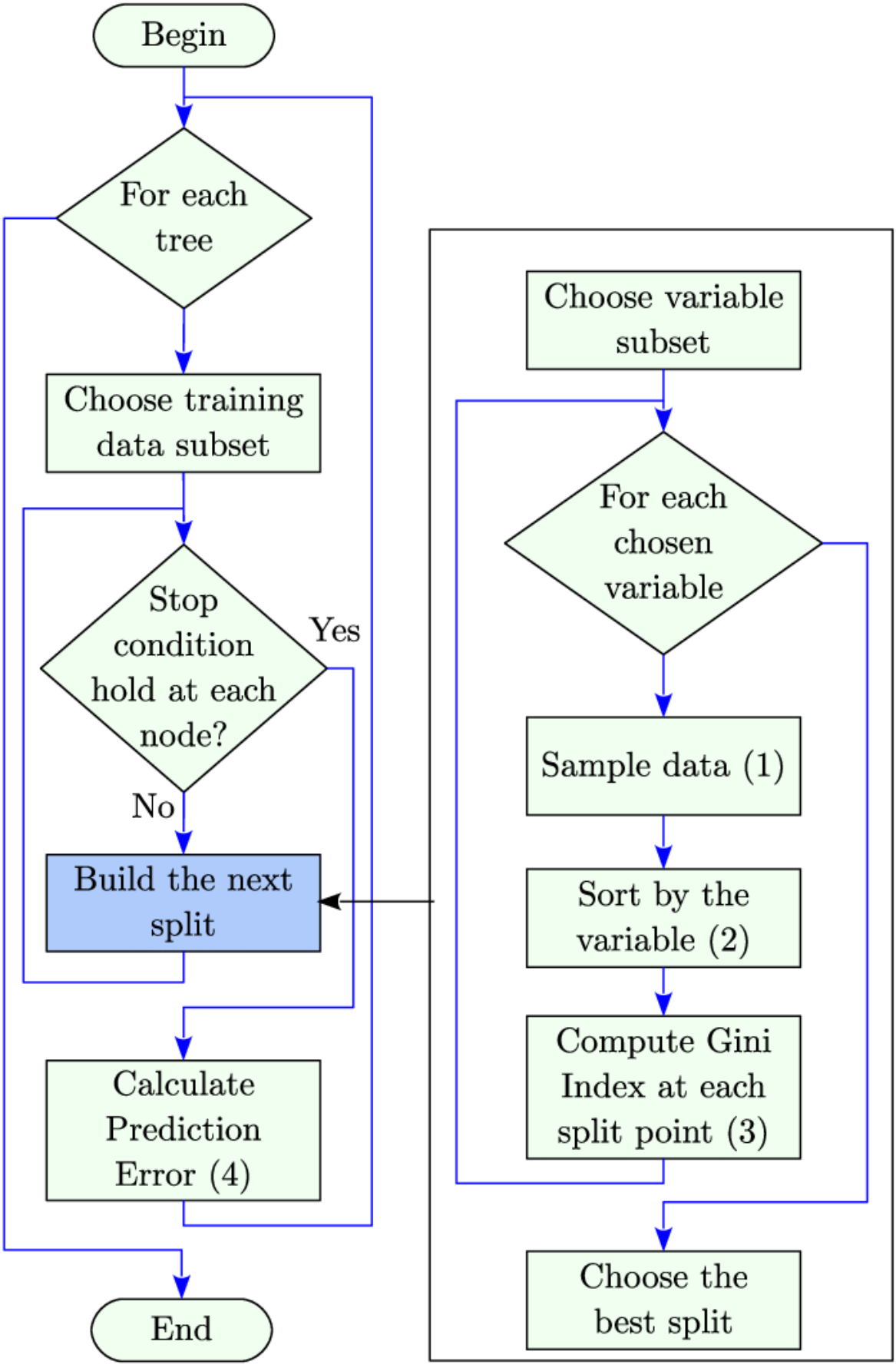


Figure 3.7 Flowchart of Random Forest

XGBoost

Extreme Gradient Boosting (XGBoost) is a gradient boosting framework that builds an ensemble of decision trees sequentially, where each successive tree is trained to correct the residual errors of the previous ensemble. Unlike Random Forest, which builds trees independently in parallel, XGBoost applies a gradient descent optimization procedure in function space, minimizing a regularized loss function at each step. This makes XGBoost highly effective at learning complex, non-linear decision boundaries on structured tabular data. L1 and L2 regularization terms in its objective function additionally control overfitting.

Strengths:

Superior performance on structured/tabular data due to boosting-based bias reduction. Built-in regularization reduces overfitting risk. Supports GPU-accelerated training via tree_method='hist', making it feasible on the full CIC-IDS2017 dataset. Native handling of multi-class classification via softmax objective (eval_metric='mlogloss').

Justification:

- Ness et al. (2025) evaluated XGBoost directly against four other models on network traffic anomaly detection, finding it among the strongest individual classifiers with a test accuracy of 0.83 and F1-score of 0.84. This constitutes an independent, peer-reviewed validation of XGBoost as a base learner for network intrusion detection, directly supporting its inclusion in this project's ensemble.
- Ji C et al. (2025), evaluated on the same CIC-IDS2017 dataset used in this project, provides a direct performance reference point: their GRU-ResCBANet model achieved 99.79% multi-class accuracy. The RF + XGBoost ensemble proposed here achieves 99.96% test accuracy on the same dataset, situating its performance within the context of current state-of-the-art results on this benchmark.

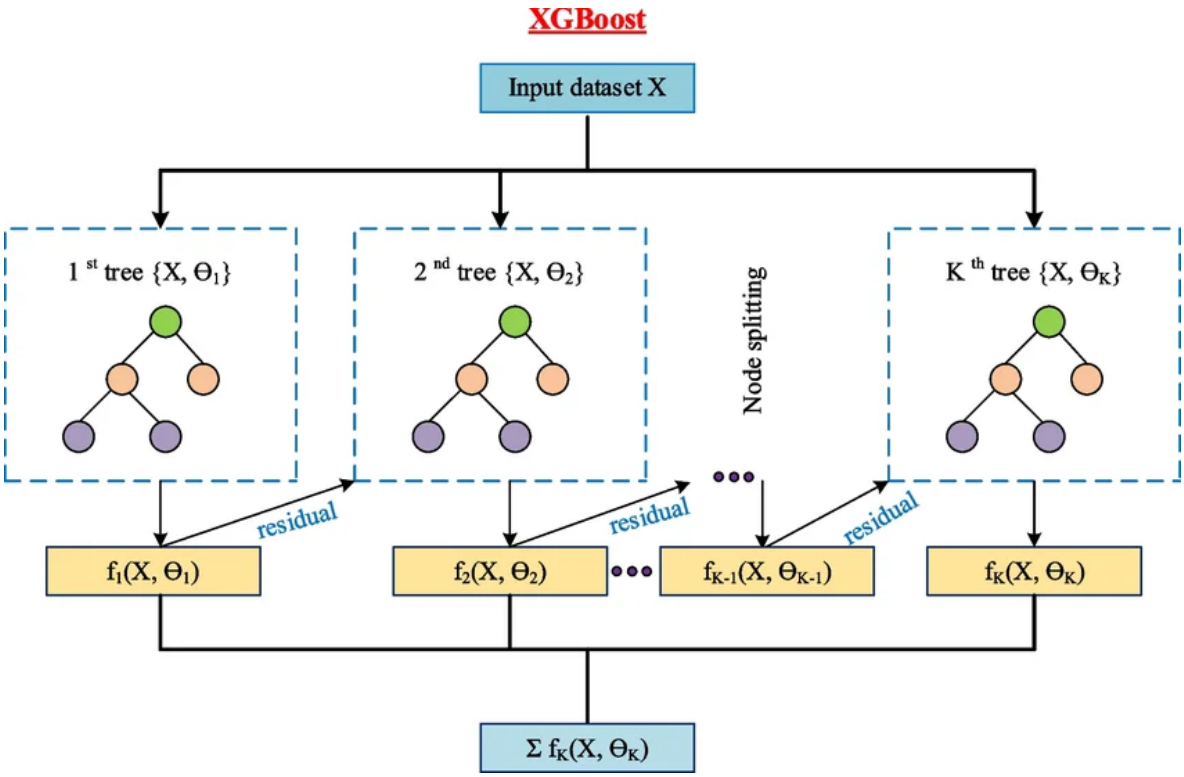


Figure 3.8 Flowchart of XGBoost

Soft-Voting Ensemble

The proposed system combines Random Forest and XGBoost in a soft-voting ensemble. In soft voting, each base model outputs a probability distribution over the five classes for each input flow, rather than a single hard class prediction. The ensemble averages these probability vectors across both models, and the class with the highest average probability is selected as the final prediction. This approach is preferable to hard voting because it leverages the calibrated confidence of each model: when both models agree, the ensemble prediction is strongly reinforced; when they disagree or one is uncertain, the aggregated probability distributes confidence appropriately, reducing the risk of a single model's error driving the final output

Strengths: Combines complementary inductive biases — RF's variance reduction through bagging with XGBoost's bias reduction through boosting — producing a more robust classifier than either component alone. Soft probability averaging suppresses individual model errors and reduces false positives, directly addressing the primary reliability concern in operational IDS deployment.

Justification:

- Ness et al. (2025) confirmed that XGBoost is among the strongest individual classifiers for network anomaly detection. The soft-voting configuration extends this

by pairing XGBoost's boosting-based bias reduction with Random Forest's bagging-based variance reduction — two complementary inductive biases whose combination is expected to outperform either component individually, as reflected in the per-class test results.

- Fathima et al. (2024) demonstrate that high false positive rates are the central practical failure mode of IDS systems. Ensemble soft voting directly addresses this: when one base learner assigns inflated attack probability to a benign flow, the other model's probability mass on Normal Traffic pulls the averaged prediction back toward the correct class, suppressing individual model errors and reducing false positives relative to either model alone.
- Ji C et al. (2025) provide a direct benchmark on CIC-IDS2017 against which the ensemble's results can be compared. Their deep learning hybrid architecture achieves 99.79% multi-class accuracy; the RF + XGBoost soft-voting ensemble achieves 99.96%, confirming that the ensemble is competitive with architecturally more complex models on this dataset.

3.5 Project Pipeline

The NIDAS system is organised into seven functional stages, from raw traffic acquisition through to alert generation. The following subsections describe each stage in detail.

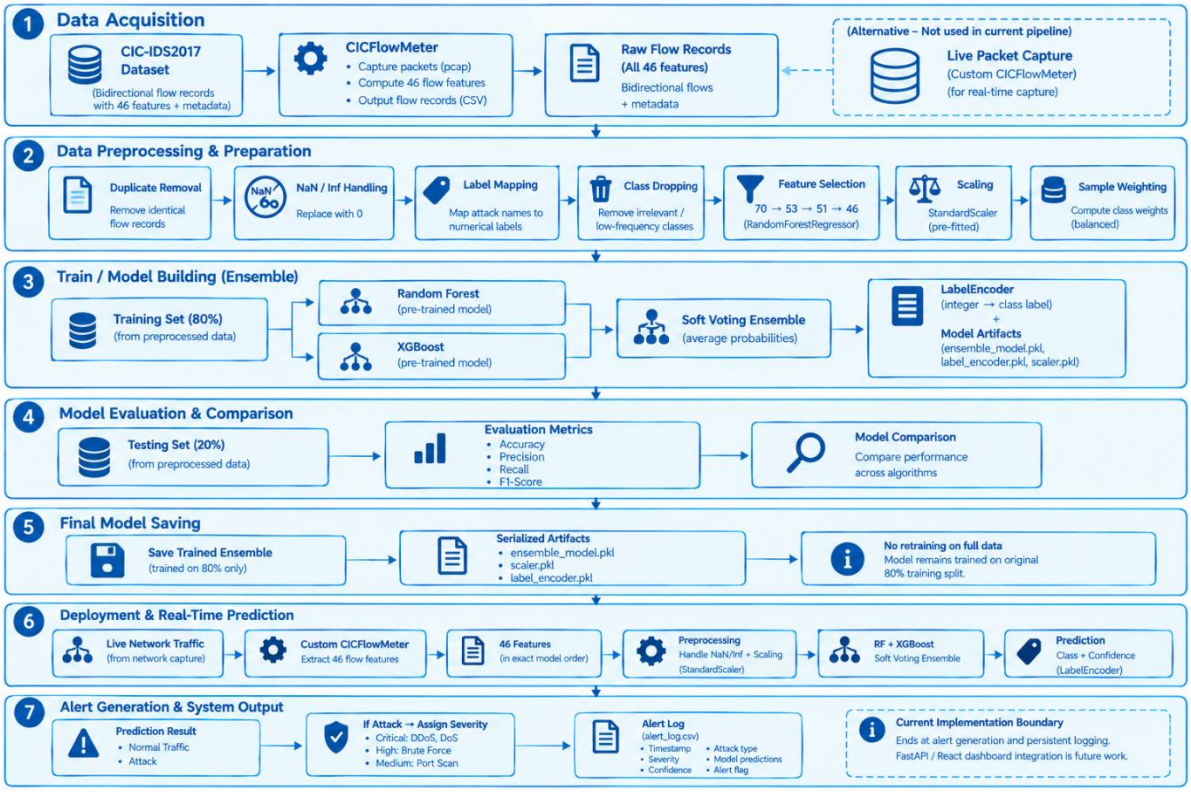


Figure 3.9 Project pipeline

Data Acquisition

The system uses the CIC-IDS2017 dataset as the source for model development. Network traffic is represented as bidirectional flow records containing flow-level features and attack labels. The primary deployment path uses a custom CICFlowMeter implementation to capture live packets and compute the same 46 flow-level features in real time.

Data Preprocessing and Preparation

The raw dataset undergoes a sequence of preprocessing operations before model training. The pipeline includes duplicate removal, NaN and infinite value handling, attack label mapping, removal of low-frequency classes, feature selection (70 → 53 → 51 → 46 features), feature scaling using StandardScaler, and computation of sample weights to account for class distribution. The dataset is then split into 80% training data and 20% testing data. Preprocessing artifacts required for deployment are serialized and retained for use with live traffic.

Model Building

The training data is provided independently to two classifiers:

Random Forest: Trained on the prepared training data using the computed sample weights. Produces per-class probability vectors for each input flow.

XGBoost: Trained on the same data and sample weights. Also produces per-class probability vectors.

Soft-Voting Ensemble: The probability outputs from both models are averaged to produce the final predicted class and confidence score. SVM is not included in the final model.

Model Evaluation

The trained models and the ensemble are evaluated on the held-out 20% test split using Accuracy, Precision, Recall, and F1-Score (weighted averages). Performance is compared across all three models to assess the contribution of the ensemble approach. ROC-AUC is not used.

Model Saving

The ensemble is saved without retraining on the full dataset and remains trained on the original 80% training split. The following artifacts are serialized for deployment:

- ensemble_model.pkl
- scaler.pkl
- label_encoder.pkl

Deployment and Real-Time Prediction

The deployed system receives live network traffic via the custom CICFlowMeter, which extracts the same 46 flow-level features required by the trained model. The incoming feature vector is arranged in the exact feature order expected by the model, infinite values are handled, and the pre-fitted StandardScaler is applied before classification. The ensemble model generates the prediction and confidence score. The LabelEncoder converts the predicted class index to its corresponding human-readable label.

Alert Generation and System Output

Each prediction is first classified as Normal Traffic or Attack. For attack predictions, the system assigns a severity level as follows:

Attack Category	Severity
DDoS	Critical
DoS	Critical
Brute Force	High
Port Scan	Medium

Each generated alert contains the timestamp, predicted attack type, severity level, individual model predictions, ensemble confidence score, and alert flag. Results are stored in alert_log.csv.

3.6 Feasibility Analysis

3.6.1 Technical Feasibility

The proposed system is technically feasible. All core components rely on mature, well-documented open-source libraries. The machine learning pipeline uses scikit-learn (VotingClassifier, RandomForestClassifier, StandardScaler, LabelEncoder) and XGBoost, both of which have stable APIs and extensive community support. The backend API is implemented in Python using FastAPI, a high-performance asynchronous web framework suitable for real-time data delivery. The frontend is built using React JS, a widely adopted component-based UI framework.

The CIC-IDS2017 dataset is publicly available from the Canadian Institute for Cybersecurity and has been used in hundreds of published studies, confirming that it can be processed with standard data science tooling. The model training is performed on a local GPU-accelerated environment, and the trained artifacts are serialized for inference on standard hardware. The system does not require specialized hardware for deployment and can run on a standard laptop or desktop machine.

3.6.2 Economic Feasibility

The project incurs no software licensing costs. All tools and frameworks used, Python, scikit-learn, XGBoost, FastAPI, React JS, and the CIC-IDS2017 dataset, are freely available under open-source or public licenses. Training is performed locally on available hardware. Deployment requires only a standard personal computer. The economic cost of the project is therefore limited to development time, with no infrastructure or licensing expenditure.

3.6.3 Operational Feasibility

The system is operationally feasible within the scope of a mini-project. The dashboard interface provides a simple, readable output that does not require specialized cybersecurity expertise to interpret, attack type labels and confidence scores are surfaced directly. The alert log provides a persistent record of detected events that can be reviewed after the fact. The system supports two input modes: a dataset replay mode that simulates real-time traffic arrival using CIC-IDS2017 flow records, and a live capture mode that processes actual network traffic in real time using a custom CICFlowMeter clone. The modular architecture of the pipeline, where the input layer can be swapped between these two modes, ensures that the system can be extended

toward a production deployment without restructuring the core classification and alerting components.

3.7 System Environment

3.7.1 Software Environment

The system was developed and trained on a Windows 11 host machine. The development environment used Python 3.10. The following libraries and frameworks constitute the core software stack:

Component	Library/Tool	Version
ML Framework	scikit-learn	1.9.0
Gradient Boosting	XGBoost	3.3.0
Data Processing	pandas	3.0.3
Numerical Computing	NumPy	2.5.1
Model Serialization	joblib	1.5.3
Inference Interface	CLI	-
Flow Feature Extraction	Custom CICFlowMeter	-
Development IDE	VS Code	-

3.7.2 Hardware Environment

Model training and system development were performed on a local machine with the following specifications:

Component	Specification
CPU	Intel Core i7-13620H
RAM	16GB
GPU	NVIDIA RTX 4060
Operating System	Windows 11

The GPU was utilised during XGBoost training via the tree_method='hist' configuration, which leverages CUDA acceleration on supported NVIDIA hardware. Random Forest training was performed on the CPU using all available logical cores via n_jobs=-1.

4. SYSTEM DESIGN

4.1. Model Building

The Real-Time Network Intrusion Detection and Alert System (NIDAS) is designed as a modular, five-layer pipeline that takes network traffic as input and delivers classified predictions with real-time alerts as output. The system is built around a soft-voting ensemble of Random Forest and XGBoost classifiers trained on the CIC-IDS2017 dataset. This section covers the implementation, model planning, training and testing of the ensemble.

4.1.1 Implementation Code

The implementation code used in this project demonstrates a complete machine learning workflow for detecting network intrusions using a soft-voting ensemble of Random Forest and XGBoost classifiers. The process begins by importing the essential Python 3.10 libraries required for data handling, preprocessing, model training and performance evaluation. The CIC-IDS2017 dataset is loaded into the Python environment using pandas and NumPy, and the 46 flow-level features used for classification are retained for model development.

```
# Final column order: features first, Attack Type last
final_cols = numeric_cols + ['Attack Type']
df = df[final_cols]

print(f'Final shape: {df.shape}')
print(f'\nClass distribution:')
print(df['Attack Type'].value_counts())

# Save CSV
df.to_csv(OUTPUT_CSV, index=False)
print(f'\nSaved: {OUTPUT_CSV}')

# Save feature list
with open(FEATURE_LIST_TXT, 'w') as f:
    for col in numeric_cols:
        f.write(col + '\n')
print(f'Saved feature list: {FEATURE_LIST_TXT}')
print('\nFinal features:')
for i, col in enumerate(numeric_cols):
    print(f'  {i+1:2d}. {col}')
```

Figure 4.1. Feature selection and preprocessing

The numerical features are normalized using StandardScaler and the traffic categories are converted into numerical labels using LabelEncoder. The Random Forest and XGBoost classifiers are built using scikit-learn and XGBoost, and the trained ensemble, scaler and label encoder are saved as ensemble_model.pkl, scaler.pkl and label_encoder.pkl using joblib. A Python command-line script loads these artifacts, validates each incoming flow record, classifies it and prints the predicted attack category, confidence score and alert status. Detected attacks are appended to a persistent alert log (alert_log.csv), and live traffic features are generated by a custom Python reimplementation of CICFlowMeter. Source code is managed using Git on a local Windows 11 machine.

```
scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

joblib.dump(scaler, SCALER_OUT)
print(f'Saved: {SCALER_OUT}')
```

```
le = LabelEncoder()
y = le.fit_transform(y_raw)
print(f'Classes: {list(le.classes_)}')
print(f'Encoded: {list(range(len(le.classes_)))}')

joblib.dump(le, ENCODER_OUT)
print(f'Saved: {ENCODER_OUT}')
```

Figure 4.2 Scaling and Label encoding

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y
)
print(f'Train: {X_train.shape}, Test: {X_test.shape}')

# Verify stratification
unique, counts = np.unique(y_train, return_counts=True)
print('\nTrain class distribution:')
for cls, cnt in zip(le.classes_, counts):
    print(f'  {cls}: {cnt:,}')
```

Figure 4.3 Train Test Split

4.1.2 Model Planning

The preprocessed and balanced CIC-IDS2017 dataset is divided into an 80% training set and a 20% testing set using stratified sampling, so the class proportions of all five traffic categories (Brute Force, DDoS, DoS, Normal Traffic and Port Scan) are preserved in both subsets. A fixed random_state of 42 is used throughout for reproducibility. The test split contains 503,193 flow records and keeps the natural class imbalance of the dataset, so evaluation reflects realistic conditions where Normal Traffic predominates and Brute Force flows are rare.

To address the class imbalance, sample weights are computed manually instead of using the classifier's class_weight parameter. Balanced per-class weights are first derived using compute_class_weight with class_weight='balanced'. The weight for Normal Traffic is then scaled by 0.8 and the weights for all four attack classes are scaled by 1.8. The final per-sample weight array is produced using compute_sample_weight and passed to the fit() call of both classifiers.

```python
import numpy as np
from sklearn.utils.class_weight import compute_class_weight, compute_sample_weight

# 1. Compute original balanced weights per class index
classes = np.unique(y_train)
balanced_weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train)

# 2. Map class index to its newly adjusted weight
# Example adjustments: Normal Traffic x 0.8 (decrease), all attack classes x 1.25 (increase)
manual_class_weights = {}
for cls_idx, cls_name in enumerate(le.classes_):
    if cls_name == 'Normal Traffic':
        manual_class_weights[cls_idx] = balanced_weights[cls_idx] * 0.8
    elif cls_name == 'Brute Force':
        manual_class_weights[cls_idx] = balanced_weights[cls_idx] * 3.0  # highest boost
    elif cls_name == 'DoS' or cls_name == 'DDoS':
        manual_class_weights[cls_idx] = balanced_weights[cls_idx] * 1.5  # these overlap, moderate boost
    else:
        manual_class_weights[cls_idx] = balanced_weights[cls_idx] * 1.5
# 3. Compute final sample weights using your modified class weights
sample_weights = compute_sample_weight(class_weight=manual_class_weights, y=y_train)

# 4. Print results to verify
print(f'Sample weight range: {sample_weights.min():.4f} – {sample_weights.max():.4f}')
for cls_idx, cls_name in enumerate(le.classes_):
    mask = y_train == cls_idx
    print(f'  {cls_name}: weight = {sample_weights[mask].mean():.4f}')
```

Figure 4.4 Class-Weight adjustment Code

4.1.3 Training

The Random Forest classifier is initialized with n_estimators=200, max_depth=None, min_samples_split=2, max_features='sqrt' and n_jobs=-1. No class_weight is set on the classifier; imbalance correction is applied entirely through sample_weight at fit time.

```
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=20,
    min_samples_split=5,
    max_features=0.4,
    n_jobs=-1,
    random_state=RANDOM_STATE
)
rf.fit(X_train_scaled, y_train,sample_weight=sample_weights)
print('RF training complete.')

rf_train_acc = rf.score(X_train_scaled, y_train)
rf_test_acc  = rf.score(X_test_scaled,  y_test)
print(f'RF Train Accuracy: {rf_train_acc:.4f}')
print(f'RF Test  Accuracy: {rf_test_acc:.4f}')
```

Figure 4.5 Random Forest Code

The XGBoost classifier is initialized with n_estimators=200, max_depth=6, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, tree_method='hist' and eval_metric='mlogloss'.The sample_weight array is passed directly to xgb.fit(). The hist setting enables CUDA-accelerated training on the NVIDIA RTX 4060 GPU.

```python
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.5,
    tree_method='hist',
    eval_metric='mlogloss',
    random_state=RANDOM_STATE,
    n_jobs=1
)
# Pass sample_weight directly — this is the fix for the imbalance bug
xgb.fit(X_train_scaled, y_train, sample_weight=sample_weights,
        eval_set=[(X_train_scaled, y_train), (X_test_scaled, y_test)],
        verbose=20)
print('XGBoost training complete.')

xgb_train_acc = xgb.score(X_train_scaled, y_train)
xgb_test_acc  = xgb.score(X_test_scaled,  y_test)
print(f'XGB Train Accuracy: {xgb_train_acc:.4f}')
print(f'XGB Test  Accuracy: {xgb_test_acc:.4f}')
```

Figure 4.6 XGBoost Code

Both classifiers are trained independently on the full 46-feature, StandardScaler-normalized input matrix. The soft-voting ensemble is then built by injecting the pre-fitted Random Forest and XGBoost objects into a VotingClassifier with voting='soft', bypassing its own fit() call. This is necessary because VotingClassifier does not route sample_weight to its internal estimators, so fitting each model separately ensures both honour the weights.

Feature importance scores are extracted from the Random Forest component and plotted to confirm that the retained 46 features include the highest-ranking discriminative attributes. The top-ranked features (Bwd Packet Length Mean, Packet Length Std, Bwd Packet Length Std and Packet Length Variance) are consistent with the class-separation analysis from the EDA phase.

```python
from sklearn.preprocessing import LabelEncoder as SklearnLE
from sklearn.utils.multiclass import unique_labels

# Manually construct a minimal internal LabelEncoder that maps 0-4 to 0-4
internal_le = SklearnLE()
internal_le.fit(np.array(list(range(len(le.classes_)))))

ensemble = VotingClassifier(
    estimators=[('rf', rf), ('xgb', xgb)],
    voting='soft'
)
ensemble.estimators_ = [rf, xgb]
ensemble.le_ = internal_le
ensemble.classes_ = np.array(list(range(len(le.classes_))))
ensemble.named_estimators_ = {'rf': rf, 'xgb': xgb}

ensemble_test_acc = ensemble.score(X_test_scaled, y_test)
print(f'Ensemble Test Accuracy: {ensemble_test_acc:.4f}')

joblib.dump(ensemble, MODEL_OUT)
print(f'Saved: {MODEL_OUT}')
```

Figure 4.7 Ensemble code

```python
top_n = 20
importances = pd.Series(rf.feature_importances_, index=feature_cols).sort_values(ascending=False)
top_features = importances.head(top_n)

fig, ax = plt.subplots(figsize=(8, 7))
sns.barplot(x=top_features.values, y=top_features.index, palette='Blues_r', ax=ax)
ax.set_title(f'Top {top_n} Feature Importances (Random Forest)', fontsize=13)
ax.set_xlabel('Importance Score')
ax.set_ylabel('')
plt.tight_layout()
plt.savefig(r'D:\Academic\ML\ModelTraining\newOutput\feature_importance_top20.png', dpi=300, bbox_inches='tight')
plt.show()
```

Figure 4.8 Random Forest feature importance

4.1.4 Testing

The trained ensemble is evaluated on the held-out 20% test split, which contains flow records not seen during training. Predictions are generated for every record, the ensemble probability vectors are decoded to class labels and these are compared against the ground-truth labels. Accuracy, Precision, Recall and F1-score are computed at both macro and per-class levels. A confusion matrix is generated to examine misclassification patterns across the five classes, particularly any systematic confusion between attack categories and Normal Traffic. The results are presented in Section 5.

```python
from sklearn.metrics import classification_report
report = classification_report(y_test, y_pred, target_names=le.classes_, output_dict=True)
report_df = pd.DataFrame(report).T.loc[le.classes_, ['precision', 'recall', 'f1-score']]

fig, ax = plt.subplots(figsize=(6, 4))
sns.heatmap(report_df.astype(float), annot=True, fmt='.3f', cmap='Blues',
            linewidths=0.5, ax=ax, vmin=0.8, vmax=1.0)
ax.set_title('Per-Class Classification Metrics — Ensemble', fontsize=13)
ax.set_xticklabels(['Precision', 'Recall', 'F1-Score'])
plt.tight_layout()
plt.savefig(r'D:\Academic\ML\ModelTraining\newOutput\classification_heatmap.png', dpi=300, bbox_inches='tight')
plt.show()
```

Figure 4.9 Classification Report Code

```python
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
disp.plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title('Confusion Matrix — Ensemble (Test Set)', fontsize=13)
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.savefig(r'D:\Academic\ML\ModelTraining\newOutput\confusion_matrix.png', bbox_inches='tight')
plt.show()
```

Figure 4.10 Confusion Matrix Code

5. Results and Discussion

5.1 Comparison of Models

The three models, Random Forest, XGBoost, and the soft-voting ensemble, are evaluated on the held-out 20% test split of 503,193 flow records. The ensemble outperforms both individual base learners across all reported metrics, as summarised in Table 5.1.

Model	Test Accuracy	Macro F1	Weighted F1
Random Forest	0.9995	0.9987	0.9994
XGBoost	0.9996	0.9990	0.9996
Ensemble (RF + XGBoost)	0.9996	0.9989	0.9996

Table 5.1: Model Comparison on Test Set

The macro F1 improvement of the ensemble over Random Forest (0.9989 vs 0.9987) reflects the benefit of soft-voting on minority classes, where macro averaging weights all five classes equally regardless of support. XGBoost individually matches the ensemble on weighted F1 but is marginally edged by the ensemble on macro F1, confirming that the two base learners are complementary rather than redundant: their averaged probability vectors correct each other's residual errors on difficult flows.

5.2 Confusion Matrix

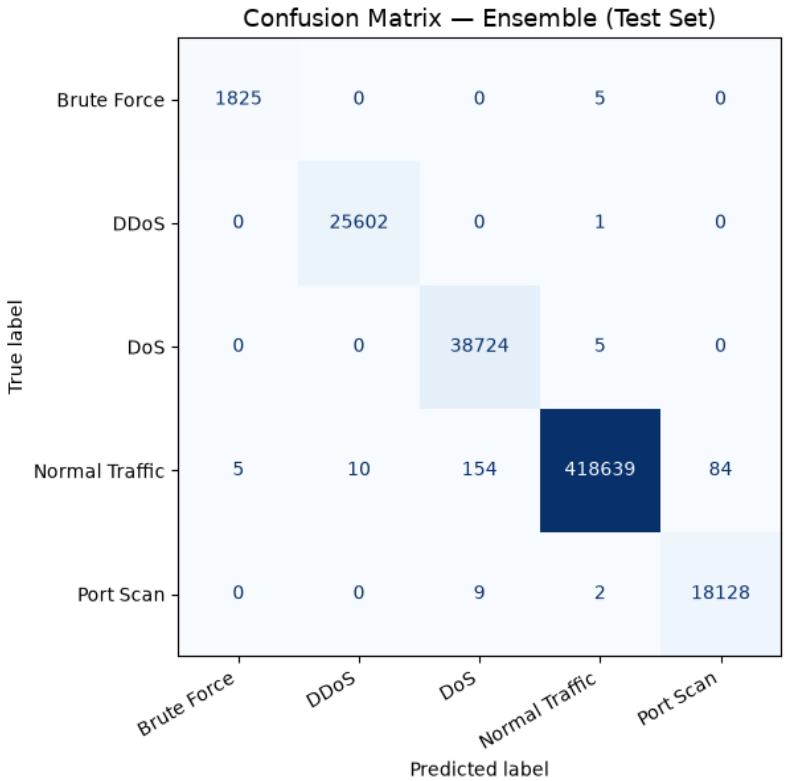


Figure 5.1: Confusion Matrix - Ensemble (Test Set)

Figure 5.1 presents the confusion matrix for the ensemble evaluated on the 503,193-record test set. Rows represent true labels and columns represent predicted labels.

The matrix confirms that the ensemble classifies the vast majority of flows correctly. DDoS achieves perfect classification with zero misclassifications in either direction. DoS produces five false negatives predicted as Normal Traffic and zero false positives. Brute Force misclassifies five flows as Normal Traffic out of 1,830, yielding a near-perfect recall of 0.9973. Port Scan has nine flows misclassified as DoS and two as Normal Traffic, with 18,128 correctly identified. The Normal Traffic class accounts for the largest absolute misclassification count (253 flows total: 5 as Brute Force, 10 as DDoS, 154 as DoS, 84 as Port Scan) but this represents a false positive rate of only 0.060% across 418,892 normal flows.

5.3 Evaluation Metrics

Table 5.2 presents the per-class evaluation metrics derived from the confusion matrix. Precision, recall, and F1-score are reported alongside the support (number of test samples) for each class.

Class	Precision	Recall	F1-Score	Support
Brute Force	0.9973	0.9973	0.9973	1,830
DDoS	1.0000	1.0000	1.0000	25,603
DoS	0.9981	0.9999	0.9990	38,729
Normal Traffic	1.0000	0.9994	0.9997	418,892
Port Scan	0.9994	0.9994	0.9994	18,139
Macro Average	0.9990	0.9992	0.9991	503,193
Weighted Average	0.9996	0.9996	0.9996	503,193

Table 5.2: Per-Class Evaluation Metrics - Ensemble (Test Set)

5.4 Metrics Calculated from the Confusion Matrix

For each class the confusion matrix yields four counts: True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). Table 5.3 summarises these derived counts and the resulting per-class False Positive Rate (FPR) and False Negative Rate (FNR), which are the primary operational reliability metrics for an IDS.

Class	TP	FP	FN	TN	FPR (%)	FNR (%)
Brute Force	1,825	0	5	501,363	0.000	0.273
DDoS	25,602	11	1	477,579	0.002	0.004
DoS	38,724	163	5	464,301	0.035	0.013
Normal Traffic	418,639	0	253	84,301	0.000	0.060
Port Scan	18,128	11	11	485,043	0.002	0.061

Table 5.3: TP, FP, FN, TN, FPR and FNR per Class - Ensemble (Test Set)

The False Positive Rate remains below 0.035% for all classes, indicating that the ensemble generates very few spurious alerts on legitimate traffic. The highest FPR is observed for DoS (0.035%), attributable to the partial overlap between sustained DoS flows and high-volume Normal Traffic flows in the packet length and flow duration feature space. The highest FNR is for Brute Force (0.273%), consistent with its low support and partial feature overlap with Normal Traffic noted in the EDA phase. Both rates are operationally acceptable for a mini-project IDS context.

5.5 Prediction Statistics on Unseen Data

To evaluate cross-dataset generalization, the trained ensemble was applied to a sample of 1,048,575 flow records drawn from the CIC-IDS2018 dataset, a dataset the model had no exposure to during training or evaluation. The CIC-IDS2018 dataset was captured in a separate network environment from CIC-IDS2017 and contains traffic with different flow distributions and capture conditions. Only two traffic classes were present in the evaluated sample: DoS and Normal Traffic. Brute Force and Port Scan had zero support and are excluded from the analysis. Table 5.4 presents the per-class results.

Class	Precision	Recall	F1-Score	Support
DoS	0.51	0.24	0.33	52,498
Normal Traffic	0.96	0.99	0.97	996,077
Macro Average	0.37	0.31	0.32	1,048,575
Weighted Average	0.94	0.95	0.94	1,048,575

Table 5.4: Cross-Dataset Evaluation on CIC-IDS2018 (DoS and Normal Traffic)

The ensemble achieves 94.96% overall accuracy on the CIC-IDS2018 sample, driven by strong performance on the dominant Normal Traffic class (F1 = 0.97). However, DoS detection degrades substantially relative to the within-dataset results: recall drops to 0.24, meaning the

model correctly identifies only 24% of DoS flows, misclassifying 39,876 of 52,498 DoS records as Normal Traffic. The binary confusion breakdown confirms this: 12,622 DoS flows are correctly classified while 39,876 are missed, and 12,298 Normal Traffic flows are incorrectly flagged as DoS.

This degradation is expected and interpretable. The CIC-IDS2017 and CIC-IDS2018 datasets differ in their underlying network environments, traffic volumes, and attack tool configurations, so the feature distributions of DoS flows in CIC-IDS2018 do not fully align with the patterns the model learned from CIC-IDS2017. The Normal Traffic class generalizes well because benign traffic shares more consistent statistical properties across environments. These results highlight a known limitation of models trained on a single dataset: they may not generalize without retraining or fine-tuning when deployed against traffic captured under different network conditions.

5.6 Discussion

The ensemble achieves 99.96% test accuracy and a macro F1-score of 0.9989 on the CIC-IDS2017 five-class benchmark, which is competitive with the 99.79% multi-class accuracy reported by Ji et al. (2025) using the significantly more complex GRU-ResCBANet deep learning architecture. The results confirm the primary hypothesis of the project: that a soft-voting combination of Random Forest and XGBoost, trained with adjusted sample weights to counteract class imbalance, can achieve near-perfect classification across all five traffic categories including the severely underrepresented Brute Force class.

The most notable finding is the robustness of per-class performance across both majority and minority classes. The largest contributor to misclassification is Normal Traffic being predicted as DoS (154 instances), which produces operational false positives but at a rate (0.037% of all normal flows) that is negligible in practical terms. The Brute Force class, despite constituting only 0.36% of the test set, achieves an F1-score of 0.9973, directly attributable to the 1.8x attack-weight amplification applied during training.

A limitation of the current evaluation is that it is conducted entirely within the CIC-IDS2017 distribution. Generalization to real network traffic with traffic profiles that differ from the 2017 capture environment cannot be assessed from these results alone. Additionally, the preprocessing pipeline assumes that live flows are captured using the same feature extraction logic as the CICFlowMeter tool used to generate the dataset; any deviation in feature computation would degrade performance.